

Q-1. Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

A.

→ CREATE DATABASE school_db;

→ CREATE TABLE students(student_id int PRIMARY KEY AUTO_INCREMENT,student_name varchar(50),age int NOT null,class varchar(10),address varchar(100));

Q-2.Insert five records into the students table and retrieve all records using the SELECT statement.

A.

→ INSERT INTO students(student_id,student_name,age,class,address) VALUES

('1','Araadhya Pattnaik','14','6th','Dombivli.Mumbai'),

('2','Bivek Dubey','16','8th','Thaane.Mumbai'),

('3','Shreya Modi','12','4th','Pandesar.Surat'),

('4','Anuska Pradhan','19','11th','Chandni-chowk.Delhi'),

('5','Atala Bihari Rauta','20','12th','Harinagar.Surat');

→ SELECT * FROM `students` ;

Q-3.Write SQL queries to retrieve specific columns (student_name and age) from the students table.

A.

→ SELECT student_name , age FROM `students`;

Q-4.Write SQL queries to retrieve all students whose age is greater than 10.

A.

→ SELECT * FROM students WHERE age>10;

Q-5.Create a table teachers with the following columns: teacher_id (Primary Key), teacher_name (NOT NULL), subject (NOT NULL), and email (UNIQUE).

A.

→ CREATE TABLE teachers(teacher_id int PRIMARY KEY,teacher_name varchar(50) NOT null,subject varchar(30) NOT null,email varchar(50) UNIQUE);

Q-6. Implement a FOREIGN KEY constraint to relate the teacher_id from the teachers table with the students table.

A.

→ ALTER TABLE students ADD teacher_id int;

→ ALTER TABLE students ADD CONSTRAINT FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)

Q-7.Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key

A.

→ CREATE TABLE courses(course_id int PRIMARY KEY,course_name varchar(50),course_credits varchar(50));

Q-8.Use the CREATE command to create a database university_db.

A.

→ CREATE DATABASE university_db;

Q-9.Modify the courses table by adding a column course_duration using the ALTER command.

A.

→ ALTER TABLE courses ADD course_duration varchar(50);

Q-10. Drop the course_credits column from the courses table.

A.

→ ALTER TABLE courses DROP course_credits;

Q-11. Drop the teachers table from the school_db database.

A.

→ DROP TABLE teachers;

Q-12. Drop the students table from the school_db database and verify that the table has been removed.

A.

→ DROP TABLE students;

Q-13. Insert three records into the courses table using the INSERT command

A.

→ INSERT INTO courses(course_id,course_name,course_duration)
VALUES
('101','JAVA','6 Months'),
('102','Reactnode','8 Months'),
('103','Python','12 Months');

Q-14. Update the course duration of a specific course using the UPDATE command.

A.

→ UPDATE courses SET course_duration = '9 Months'
WHERE course_id = '102';

Q-15.Delete a course with a specific course_id from the courses table using the DELETE command.

A.

→ DELETE FROM courses WHERE course_id = 101;

Q-16. Retrieve all courses from the courses table using the SELECT statement.

A.

→ SELECT * FROM courses ;

Q-17.Sort the courses based on course_duration in descending order using ORDER BY.

A.

→ SELECT * FROM courses ORDER BY course_duration DESC;

Q-18.Limit the results of the SELECT query to show only the top two courses using LIMIT.

A.

→ SELECT * FROM courses LIMIT 2;

Q-19. Create two new users user1 and user2 and grant user1 permission to SELECT from the courses table.

A.

→ GRANT SELECT ON courses TO user1;

→ SELECT *FROM courses;

Q-20.Revoke the INSERT permission from user1 and give it to user2.

A.

→ REVOKE INSERT ON courses FROM user1;
→ GRANT INSERT ON courses TO user2;

Q-21.Insert a few rows into the courses table and use COMMIT to save the changes.

A.

→ INSERT INTO courses(course_id,course_name,course_duration)
VALUES
(101,'JAVA','6 Months'),
(102,'Reactnode','8 Months'),
(103,'Python','12 Months'),
(104,'HTML','3 Months');
COMMIT;

Q-22.Insert additional rows, then use ROLLBACK to undo the last insert operation.

A.

→ INSERT INTO courses(course_id,course_name,course_duration)
VALUES
(101,'JAVA','6 Months'),
(102,'Reactnode','8 Months'),
(103,'Python','12 Months'),
(104,'HTML','3 Months');
ROLLBACK;

Q-23.Create a SAVEPOINT before updating the courses table, and use it to roll back specific changes.

A.

→ START TRANSACTION;
SAVEPOINT sp_before_update;
UPDATE courses
SET course_duration = course_duration + 2 Months
WHERE course_id = 101;
ROLLBACK TO sp_before_update;

Q-24. Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

A.

```
→ CREATE TABLE departments (  
dept_id INT PRIMARY KEY,  
dept_name VARCHAR(50)  
);  
CREATE TABLE employees (  
emp_id INT PRIMARY KEY,  
emp_name VARCHAR(50),  
dept_id INT,  
emp_salary DOUBLE,  
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);  
SELECT * FROM departments INNER JOIN employees on  
departments.dept_id = employees.dept_id;
```

Q-25. Use a LEFT JOIN to show all departments, even those without employees.

A.

```
→ SELECT * FROM departments LEFT JOIN employees on  
departments.dept_id = employees.dept_id;
```

Q-26. Group employees by department and count the number of employees in each department using GROUP BY.

A.

```
→ SELECT dept_name, COUNT(emp_id) AS total_employee FROM  
employees, department where employees.dept_id=department.dept_id  
GROUP BY department.dept_name;
```

Q-27. Use the AVG aggregate function to find the average salary of employees in each department.

A.

```
→ SELECT dept_name, AVG(emp_salary) AS avg_salary FROM  
employees, department where employees.dept_id=department.dept_id  
GROUP BY department.dept_name;
```

Q-28. Write a stored procedure to retrieve all employees from the employees table based on department.

A.

```
→ CREATE PROCEDURE getEmployeesByDepartment  
    @dept_name Varchar(20)  
    AS  
    BEGIN  
        SELECT  
            emp_id,  
            emp_name,  
            dept_name,  
            emp_salary  
        FROM employees e  
        INNER JOIN departments d  
        ON employees.dept_id = department.dept_id  
        WHERE employees.dept_name = @dept_id;  
    END;;
```

```
CALL getEmployeesByDepartment 'IT';
```

Q-29. Write a stored procedure that accepts course_id as input and returns the course details.

A.

```
→ CREATE PROCEDURE getCourseDetails  
    @course_id INT  
    AS  
    BEGIN  
        SELECT * FROM courses
```

```
WHERE course_id = @course_id  
END;
```

Q-30.Create a view to show all employees along with their department names.

A.

```
→ CREATE VIEW employee_department AS  
    SELECT emp_id,emp_name,dept_name,emp_salary FROM employees  
    INNER JOIN departments on employees.dept_id = departments.dept_id;
```

Q-31.Modify the view to exclude employees whose salaries are below \$50,000.

A.

```
→ CREATE OR REPLACE VIEW employee_department AS  
    SELECT emp_id,emp_name,dept_name,emp_salary FROM employees  
    INNER JOIN departments on employees.dept_id = departments.dept_id  
    WHERE emp_salary>=50000;
```

Q-32.Create a trigger to automatically log changes to the employees table when a new employee is added.

A.

```
→ CREATE TABLE employee_log (  
log_id INT AUTO_INCREMENT PRIMARY KEY,  
emp_id INT,  
emp_name VARCHAR(50),  
action_type VARCHAR(20),  
action_time DATETIME  
);  
→DELIMITER//  
CREATE TRIGGER employee_new AFTER INSERT ON employees  
    FOR EACH ROW  
    BEGIN  
        INSERT INTO employee_log (emp_id, action_time)  
VALUES (NEW.emp_id, NOW());
```

```
END;  
DELIMITER;
```

Q-33.Create a trigger to update the last_modified timestamp whenever an employee record is updated.

A.

```
→ ALTER TABLE employees  
ADD COLUMN last_modified DATETIME;
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_employee_last_modified  
BEFORE UPDATE ON employees  
FOR EACH ROW  
BEGIN  
SET NEW.last_modified = NOW();  
END //
```

```
DELIMITER ;
```

Q-34.Write a PL/SQL block to print the total number of employees from the employees table.

A.

```
→ DECLARE  
    total NUMBER;  
BEGIN  
    SELECT COUNT(*)  
    INTO total  
    FROM employees;  
  
    DBMS_OUTPUT.PUT_LINE('Total Employees : ' || total);  
END;  
/
```

Q-35.Create a PL/SQL block that calculates the total sales from an orders table.

A.

```
→ DECLARE
    total_sales NUMBER;
BEGIN
    SELECT SUM(order_amount)
    INTO total_sales
    FROM orders;

    DBMS_OUTPUT.PUT_LINE('Total Sales : ' || total_sales);
END;
/
```

Q-36.Write a PL/SQL block using an IF-THEN condition to check the department of an employee.

A.

```
→ DECLARE
    dept_id employees.dept_id%TYPE := 2;
    dept_name departments.dept_name%TYPE;
BEGIN
    SELECT departments.dept_name
    INTO dept_name
    FROM employees JOIN departments ON employees.dept_id =
    department.dept_id WHERE employees.dept_id = dept_id
    AND ROWNUM = 1;
    IF dept_name = 'Finance' THEN
        DBMS_OUTPUT.PUT_LINE('Employee belongs to Finance
        department');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Employee belongs to ' || dept_name || '
        department');
    END IF;
END;
/
```

Q-37. Use a FOR LOOP to iterate through employee records and display their names.

A.

```
→ BEGIN
FOR rec IN (SELECT emp_name FROM employees) LOOP
DBMS_OUTPUT.PUT_LINE(rec.emp_name);
END LOOP;
END;
/
```

Q-38. Write a PL/SQL block using an explicit cursor to retrieve and display employee details.

A.

```
→ DECLARE
CURSOR emp_cur IS
SELECT emp_id, emp_name, emp_salary
FROM employees;

id employees.emp_id%TYPE;
name employees.emp_name%TYPE;
salary employees.emp_salary%TYPE;
BEGIN
OPEN emp_cur;

LOOP
FETCH emp_cur INTO v_id, v_name, v_salary;
EXIT WHEN emp_cur%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(
'ID: ' || id ||
', Name: ' || name ||
', Salary: ' || salary
);
END LOOP;
```

```
CLOSE emp_cur;  
END;  
/
```

Q-38.Create a cursor to retrieve all courses and display them one by one.

A.

```
→ DECLARE  
CURSOR course_cur IS  
SELECT course_id, course_name  
FROM courses;  
  
id courses.course_id%TYPE;  
name courses.course_name%TYPE;  
BEGIN  
OPEN course_cur;  
  
LOOP  
FETCH course_cur INTO id, name;  
EXIT WHEN course_cur%NOTFOUND;  
  
DBMS_OUTPUT.PUT_LINE('Course ID: ' || id ||  
, Course Name: ' || name);  
END LOOP;  
  
CLOSE course_cur;  
END;  
/
```

Q-39.Perform a transaction where you create a savepoint, insert records, then rollback to the savepoint.

A.

```
→ START TRANSACTION;  
  
SAVEPOINT sp_before_insert;
```

```
INSERT INTO employees (emp_id, emp_name, dept_id, emp_salary)
VALUES (201, 'Riya Mehta', 3, 50000);
```

```
INSERT INTO employees (emp_id, emp_name, dept_id, emp_salary)
VALUES (202, 'Arjun Nair', 2, 52000);
```

```
ROLLBACK TO sp_before_insert;
```

```
COMMIT;
```

Q-40.Commit part of a transaction after using a savepoint and then rollback the remaining changes.

A.

→ START TRANSACTION;

```
INSERT INTO employees (emp_id, emp_name, dept_id, emp_salary)
VALUES (301, 'Karan Patel', 1, 45000);
```

```
SAVEPOINT sp1;
```

```
INSERT INTO employees (emp_id, emp_name, dept_id, emp_salary)
VALUES (302, 'Meera Shah', 2, 48000);
```

```
ROLLBACK TO sp1;
```

```
COMMIT;
```

